

Programação Orientada a Objetos - CST em Análise e Desenvolvimento de Sistemas

Associação entre classes

Agregação, composição e dependência

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](#)

Diagrama de classes UML

Linguagem de modelagem unificada – UML

- Linguagem (notação gráfica + semântica) para especificar, construir e documentar os artefatos dos sistemas
- Formas de uso para a UML
 - **Como rascunho** – diagramas incompletos e informais (geralmente escritos em um quadro branco) para explorar partes difíceis do problema
 - **Como projeto de software** – diagramas detalhados que podem ser usados para gerar código (engenharia direta) ou foram gerados para entender um código existente (engenharia reversa)

Diagramas da UML

■ Diagramas estruturais

- Diagrama de classes
- Diagrama de objetos
- Diagrama de componentes
- Diagrama de implantação

■ Diagramas comportamentais

- Diagrama de caso de uso
- Diagrama de sequência
- Diagrama de colaboração
- Diagrama de atividades

Diagramas da UML

■ Diagramas estruturais

- Diagrama de classes
- Diagrama de objetos
- Diagrama de componentes
- Diagrama de implantação

■ Diagramas comportamentais

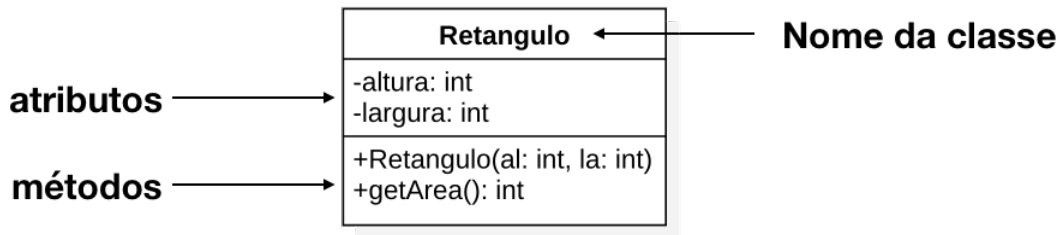
- Diagrama de caso de uso
- Diagrama de sequência
- Diagrama de colaboração
- Diagrama de atividades

Diagrama de classes

Diagrama de classes

- Representa a estrutura estática de um sistema orientado a objetos, mostrando as classes, seus atributos, métodos e os relacionamentos entre elas
 - não precisam ser completos, podem ser usados para destacar somente as partes mais importantes do sistema
- **Na fase de análise de requisitos**, é usado para modelar as entidades do domínio do problema e seus relacionamentos
- **Na fase de projeto**, é usado para modelar as classes do sistema, seus atributos, métodos e os relacionamentos entre elas
- **Na fase de manutenção**, é usado para entender o código existente e para planejar alterações no sistema

Representação de classes em um diagrama de classes UML



- Modificadores de acesso (+, -, #)
- Membros estáticos ficam sublinhados
- Métodos triviais (*getters* e *setters*) devem ser omitidos para manter o diagrama mais legível

Modelagem de classes

Dado
valorDaFace

Perspectiva conceitual

Dado
- valorDaFace : int
+ obterValorDaFace() : int

Perspectiva de implementação

- **Perspectiva conceitual**, onde as classes representam conceitos do domínio do problema
- **Perspectiva de implementação**, onde as classes representam as classes do código-fonte do sistema

Ferramentas de modelagem UML

- **Mermaid** – <https://mermaid.ai/live>
 - É uma ferramenta gratuita para criar diagramas a partir de texto
- **PlantUML** – <https://plantuml.com>, <https://plantumleditor.com/webedit/>
 - É uma ferramenta gratuita para criar diagramas a partir de texto
- **Draw.io** – <https://www.drawio.com>
 - Ferramenta gratuita e de propósito geral para criar diagramas, incluindo diagramas UML
 - Permite criar diagramas de forma visual, arrastando e soltando elementos, sem a necessidade de escrever código
- **StarUML** – <http://staruml.io>
 - Ferramenta comercial para modelagem UML, com suporte a uma ampla variedade de diagramas e recursos avançados de modelagem

Mermaid: <https://mermaid.ai/live>

Documentação <https://mermaid.js.org/syntax/classDiagram.html>

```
```mermaid
classDiagram
 direction LR

 class Retangulo{
 - int altura
 - int largura
 + Retangulo(int a, int l)
 + getArea() int
 }
...
```
```

Retangulo

- int altura
- int largura

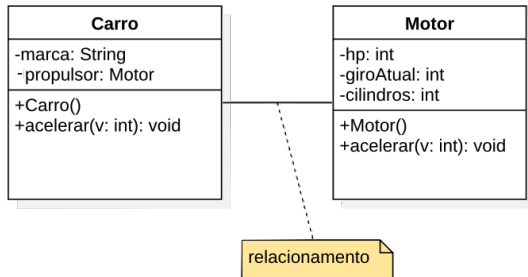
+ Retangulo(int a, int l)
+ getArea() : int

Associação entre classes

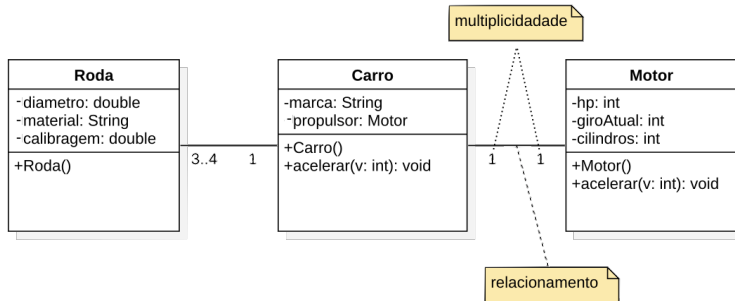
Associação entre classes

- Relacionamento entre classes que permite o compartilhamento de informação e colaboração para a execução de computação
- Descreve o vínculo entre objetos de uma ou mais classes
- A classe **Carro** possui um relacionamento com a classe **Motor**, pois um objeto da classe Carro contém 1 objeto da classe Motor

```
public class Carro{  
    private String marca;  
    private Motor propulsor;  
  
    public Carro(String m, Motor mo){  
        this.marca = m;  
        this.propulsor = mo;  
    }  
}
```



Multiplicidade em associações entre classes

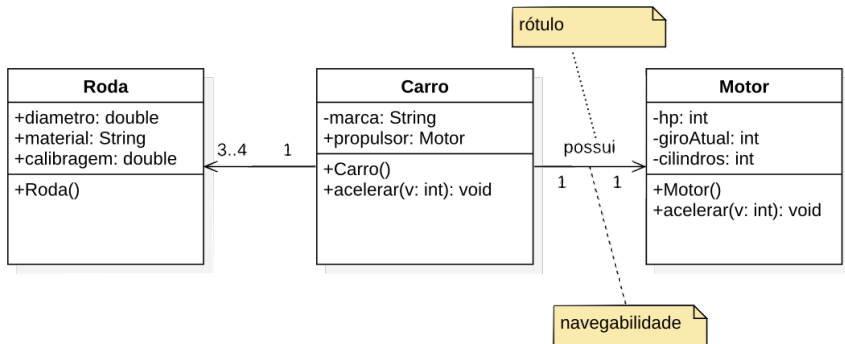


| multiplicidade | |
|----------------|---------------|
| 0..* | Zero ou mais |
| 1..* | Um ou mais |
| * | Muitos |
| 1 | Exatamente um |
| 3..4 | De 3 a 4 |

Associação unidirecional

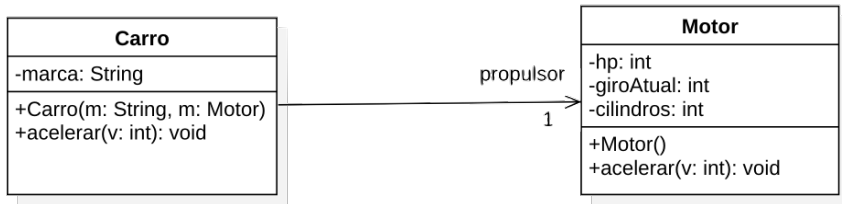
Representada por uma linha com uma seta apontando para a classe que é referenciada

- A classe **Carro** tem uma referência para um objeto da classe **Motor**, mas a classe **Motor** não tem uma referência para um objeto da classe **Carro**
- A classe **Carro** pode acessar os membros públicos do objeto da classe **Motor**, mas a classe **Motor** não pode acessar os membros públicos do objeto da classe **Carro**

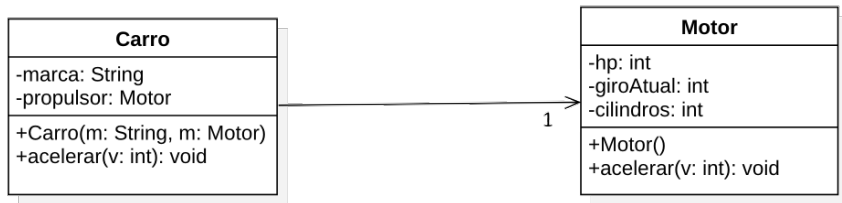


Associação: notação textual vs linha de associação

- Uso da notação de associação para indicar que Carro possui uma referência para uma instância de Motor

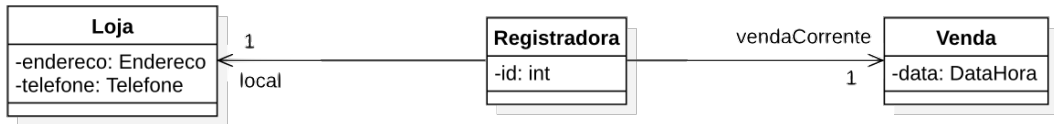


- Notação textual de atributo



Associação: notação textual vs linha de associação

Classe Registradora possui 3 atributos:
1. id
2. vendaCorrente
3. local



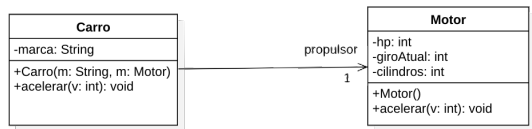
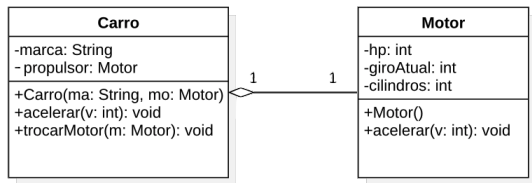
Associação do tipo Agregação

- Representa uma relação onde uma classe (todo) agrupa outras classes (partes), mas as partes podem existir independentemente do todo
- Usada quando o ciclo de vida dos objetos-parte não depende do objeto-todo.

Exemplos

- Um **carro** possui um **motor**. Se o carro deixar de existir, o motor poderia ser colocado em outro carro
- Um **livro** possui um **autor**. Se o livro deixar de existir, o autor ainda pode existir e escrever outros livros

Associação do tipo Agregação



- Representada por uma linha de associação com um losango vazio na extremidade do lado do objeto-todo
- Pode ser representada por uma linha de associação comum, sem a necessidade de usar a notação de losango para indicar a agregação

Associação do tipo Agregação

```
public class Carro{  
    private String marca;  
    private Motor propulsor;  
  
    public Carro(String m, Motor mo){  
        this.marca = m;  
        this.propulsor = mo;  
    }  
  
    public void acelerar(int valor){  
        this.propulsor.acelerar(valor);  
    }  
  
    public void trocarMotor(Motor mo){  
        this.propulsor = mo;  
    }  
}
```

- A classe **Carro** tem uma referência para um objeto da classe **Motor**
- Ao criar um objeto da classe Carro, é necessário passar um objeto da classe Motor para o construtor do Carro
- Ao destruir um objeto da classe Carro, o objeto da classe Motor pode continuar existindo se houver outras referências para ele

Associação do tipo Composição

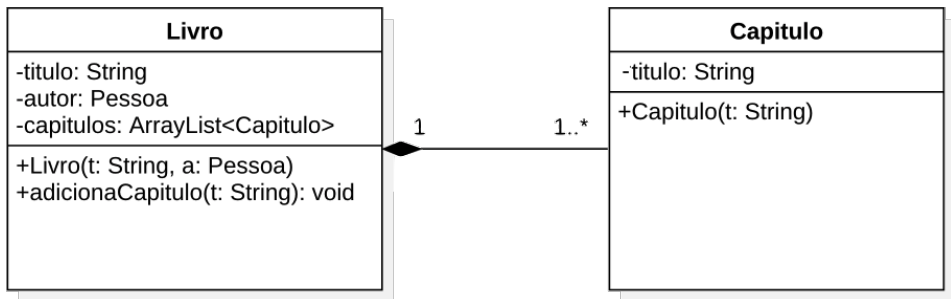
- A composição representa uma relação de “todo-parte” onde a parte pertence exclusivamente ao todo.
- Se o objeto-todo for destruído, todos os seus objetos-parte também são destruídos automaticamente.
- Os objetos-parte não podem existir independentemente do objeto-todo; sua existência está totalmente vinculada ao ciclo de vida do todo.

Exemplos

- Um objeto **Livro** é composto por diversos objetos **Capítulo**. Se destruímos um **Livro**, não faria mais sentido os **Capítulos** existirem
- Um objeto **Aluno** pode ser composto por um objeto **Endereço**. Se destruímos o objeto Aluno, não faria mais sentido o objeto Endereço existir

Associação do tipo Composição

- Representada por uma linha de associação com um losango preenchido na extremidade do lado do objeto-todo
- Indica que a classe-todo é responsável por criar e destruir os objetos-parte, ou seja, o ciclo de vida dos objetos-parte é controlado pelo objeto-todo



Associação do tipo Composição

```
public class Livro{  
    private String titulo;  
    private ArrayList<Capitulo> capitulos;  
    private Pessoa autor;  
  
    public Livro(String t, Pessoa a){  
        this.titulo = t;  
        this.capitulos = new ArrayList<>();  
        this.autor = a;  
    }  
  
    public void adicionaCapitulo(String titulo){  
        this.capitulos.add(new Capitulo(titulo));  
    }  
}
```

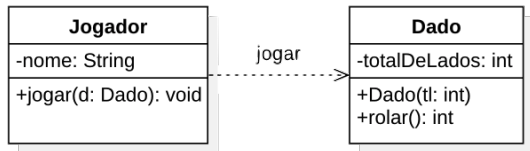
- Os objetos da classe **Capítulo** são criados e destruídos pelo objeto da classe **Livro**
- Não há referências para os objetos da classe Capítulo fora do objeto da classe Livro, ou seja, os objetos da classe Capítulo não podem existir independentemente do objeto da classe Livro

Associação do tipo Dependência

- Indica que uma classe utiliza temporariamente outra para realizar uma tarefa
- Mudanças na classe utilizada podem impactar a classe dependente, mas mudanças na classe dependente não impactam a classe utilizada

Exemplo

- O jogador precisa de um dado para obter um resultado de uma jogada, mas o dado não faz parte do jogador, ou seja, o dado não é um atributo do jogador

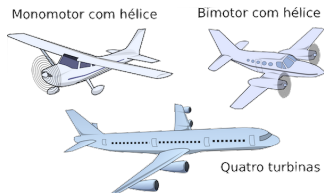


```
public class Jogador {
    private String nome;

    public void jogar(Dado dado){
        int resultado = dado.rolar();
    }
}
```


Exercício

Faça um diagrama de classes UML para um jogo



- Cada **Avião** deve possuir:
 - Número máximo de tripulantes
 - Número máximo de passageiros
 - Capacidade máxima de combustível
- Um avião pode ter de 1 a 8 **Motores**
- Cada **Motor** pode ser do tipo **turbina** ou **hélice**
 - Cada motor pode estar ligado ou desligado
- Ao ligar o avião, todos os motores devem ser ligados. Ao desligar o avião, todos os motores devem ser desligados
- É possível ligar ou desligar cada motor individualmente

Leitura obrigatória



Craig Larman

Utilizando UML e padrões

3a. Edição - Editora Bookman, 2007

- Capítulos 16, seções: 16.1 – 16.9, 16.10, 16.13



Gilleanes T.A. Guedes

UML2 – uma abordagem prática

2a. Edição - Editora Novatec, 2011

- Capítulos 4, seções: 4.1–4.2.2, 4.2.4, 4.2.5, 4.2.8

Aulas baseadas em



Craig Larman

Utilizando UML e padrões

3a. Edição - Editora Bookman, 2007